Wheat Disease Forecasting Using Elastic Net Regression

Author: Alex Rabeau
Date: April 2026

Introduction

This report presents a forecasting pipeline for predicting wheat disease (Zymoseptoria tritici and Yellow rust) severity and crop incidence across UK regions using an Elastic Net regression framework.

The workflow integrates multiple datasets, including pest observations, agronomic variables, bioclimatic indicators, fungicide usage, and land-use composition.

Key features include lagged variable construction, Kalman-based imputation, skewness correction, and region-specific modelling.

The target variables are:

- **L1_Disease_Severity**
- **L1_Crop_Incidence**
- **L2_Disease_Severity**
- **L2_Crop_Incidence**

---

1. Package and Function Loading

We begin by configuring the analysis environment, including loading the required packages, defining helper functions, and setting key modelling parameters.

```
# Import packages and functions
import pandas as pd
import numpy as np
from scipy.stats import skew

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import ElasticNetCV, ElasticNet
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import TimeSeriesSplit
from pykalman import KalmanFilter

# Set parameters
PEST_FILE = "data/pest_data.csv"
AGRONOMIC_FILE = "data/agronomic_data.csv"
BIOCLIM_FILE = "data/bioclim_data.csv"
FUNGICIDE_FILE = "data/fungicide_data.csv"
LUC_FILE = "data/prop_LUC.csv"

DISEASE_VARS = [
    "L1_Zymoseptoria_tritici_Disease_Severity",
    "L1_Yellow_rust_Disease_Severity",
    "L1_Zymoseptoria_tritici_Crop_Incidence",
    "L1_Yellow_rust_Crop_Incidence",
```

```
    "L2_Zymoseptoria_tritici_Disease_Severity",
    "L2_Yellow_rust_Disease_Severity",
    "L2_Zymoseptoria_tritici_Crop_Incidence",
    "L2_Yellow_rust_Crop_Incidence"
]

N_LAGS = 2
TRAIN_END = 2020
TEST_END = 2024
```

2. Data Loading and Feature Engineering

The core dataset is loaded into the work environment.

To capture temporal dynamics, we introduce lagged versions of all target variables. We wish to include disease variables at t-2, t-1 and t to predict disease severity and incidence at t+1.

```
# Load data
pest_data = pd.read_csv(PEST_FILE)

# Feature engineering: lagged variables
lagged = pest_data.copy()
lagged["Year"] = lagged["Year"].astype(int)
lagged = lagged.sort_values(["Region","Year"])

for col in DISEASE_VARS:
    lagged[f"{col}_lead1"] = (
        lagged.groupby("Region")[col].shift(-1)
    )

    for lag in range(1, N_LAGS+1):
        lagged[f"{col}_lag{lag}"] = (
            lagged.groupby("Region")[col].shift(lag)
        )

# Set target variables
TARGET_VARS = [f"{col}_lead1" for col in DISEASE_VARS]
```

3. Feature Integration

The core dataset is enriched by merging with agronomic, climate, fungicide, and land-use datasets.

All variables are aligned by Year and Region.

```
# Load in Explanatory Variables
agronomic_data = pd.read_csv(AGRONOMIC_FILE)
bioclim_data = pd.read_csv(BIOCLIM_FILE)
fungicide_data = pd.read_csv(FUNGICIDE_FILE)
luc_data = pd.read_csv(LUC_FILE)

# Merge datasets
FORECASTING_DF = lagged.copy()
```

```
FORECASTING_DF["Year"] = FORECASTING_DF["Year"].astype(int)

FORECASTING_DF = (
    FORECASTING_DF
    .merge(agronomic_data, on=["Year","Region"], how="left")
    .merge(bioclim_data, on=["Year","Region"], how="left")
    .merge(fungicide_data, on=["Year","Region"], how="left")
    .merge(luc_data, on=["Year","Region"], how="left")
)

for col in FORECASTING_DF.columns:
    if col != "Region":
        FORECASTING_DF[col] = pd.to_numeric(FORECASTING_DF[col])
```

4. Missing Data Imputation (Kalman Filtering)

Missing values in predictor variables are imputed using a Kalman smoothing approach, which preserves temporal structure.

```
def kalman_impute(series):

    s = series.replace([np.inf, -np.inf], np.nan)

    if s.notna().sum() < 3:
        return s

    filled = s.ffill().bfill()
    values = filled.values
    mask = np.isnan(s.values)
    kf = KalmanFilter(initial_state_mean=values[0])

    try:
        smoothed, _ = kf.em(values, n_iter=5).smooth(values)
        smoothed = smoothed.flatten()
        values[mask] = smoothed[mask]

    except:
        values[mask] = np.nanmean(values)

    return pd.Series(values, index=s.index)

disease_cols = [c for c in FORECASTING_DF.columns if "L1" in c or "L2" in c]

for col in FORECASTING_DF.columns:

    if col not in ["Year","Region"] and col not in disease_cols:

        FORECASTING_DF[col] = kalman_impute(FORECASTING_DF[col])

FORECASTING_DF = FORECASTING_DF.dropna()
```

5. Handling Skewness

Predictor variables exhibiting high skewness ($|\text{skew}| > 1$) are transformed to stabilise variance and improve model performance.

```
# Define predictors
predictors = [
    c for c in FORECASTING_DF.columns
    if c not in ["Year","Region"] + TARGET_VARS
]

# Handling skewness
skewness_results = []

for col in predictors:

    x = FORECASTING_DF[col]

    skew_val = skew(x, nan_policy="omit")

    transformation = "none"

    if abs(skew_val) > 1:

        if skew_val > 1 and (x > 0).all():

            FORECASTING_DF[col] = np.log1p(x)
            transformation = "log1p"

        elif skew_val > 1:

            FORECASTING_DF[col] = np.sqrt(x - x.min() + 1)
            transformation = "sqrt"

        elif skew_val < -1:

            FORECASTING_DF[col] = x**2
            transformation = "squared"

    skewness_results.append({
        "variable": col,
        "skewness": skew_val,
        "transformation": transformation
    })

skewness_results = pd.DataFrame(skewness_results)
```

6. Region-Specific Elastic Net Modelling

Models are trained separately for each region using an Elastic Net regression with cross-validated regularisation.

- Training period: up to 2020

- Testing period: 2021–2024
- Forecast: 2026

Model performance is evaluated using RMSE and normalised relative RMSE to allow comparison across regions and targets.

```
# Forecasting loop
forecast_results = []
performance_results = []

regions = FORECASTING_DF["Region"].unique()

for reg in regions:

    print("Processing:", reg)

    df_region = FORECASTING_DF[FORECASTING_DF["Region"]==reg]

    # Train/test split based on time
    train = df_region[df_region["Year"]<=TRAIN_END]
    test = df_region[(df_region["Year"]>TRAIN_END) & (df_region["Year"]<=TEST_END)]

    # Feature scaling
    scaler = StandardScaler()
    X_train = scaler.fit_transform(train[predictors])
    X_test = scaler.transform(test[predictors])
    tscv = TimeSeriesSplit(n_splits=5)

    for target in TARGET_VARS:

        y_train = train[target]
        y_test = test[target]

        # Elastic Net with cross-validation
        enet_cv = ElasticNetCV(l1_ratio=0.5, cv=tscv, random_state=123)

        # Fit model
        enet_cv.fit(X_train, y_train)
        model = enet_cv

        # Predict on test set
        pred_test = model.predict(X_test)

        # Model evaluation
        rmse = np.sqrt(mean_squared_error(y_test, pred_test))

        print(target.replace("_lead1", ""), "RMSE:", round(rmse,4))

        performance_results.append({
            "region": reg,
            "target": target.replace("_lead1", ""),
            "rmse": rmse
        })
```

```
        # Future forecast (2026)
        X_future = df_region[df_region["Year"]==2025][predictors]

        if not X_future.empty:

            X_future = scaler.transform(X_future)

            pred_2026 = model.predict(X_future)[0]

            forecast_results.append({
                "region": reg,
                "target": target.replace("_lead1", ""),
                "year": 2026,
                "forecast_value": float(pred_2026)
            })
```

7. Forecasting Output

```
# Export results
performance_results = pd.DataFrame(performance_results)
performance_results.to_csv("pest_model_performance.csv", index=False)

forecast_results = pd.DataFrame(forecast_results)
forecast_results.to_csv("pest_forecasts_2026.csv", index=False)
```

Conclusion

This pipeline demonstrates a robust and scalable approach for forecasting wheat disease dynamics using integrated environmental and agronomic data.

Key strengths include:

- Region-specific modelling to capture spatial patterns
- Lagged features to account for temporal dependence
- Kalman imputation to handle data gaps
- Elastic Net regularisation for stability in high-dimensional settings

The resulting forecasts provide actionable insights for agricultural planning and disease management strategies.